

React Hooks - useState 和 useEffect 只使用前端

我們將學習非常重要的 React Hooks，它們是 useState 和 useEffect Hooks。Hooks 是 React 16.8 的新增功能。它們讓您無需編寫類別即可使用狀態和其他 React 功能。

我什麼時候使用 Hook？

如果您編寫一個 Function 組件並意識到您需要向它添加一些狀態，那麼以前您必須將它轉換為一個類別組件。現在您可以在現有 Function 組件中使用 Hook。

建立 React JS 前端應用程序

我們將使用 VS Code 來建置 React 應用程序。

1 - 使用 Create React App 建立 React UI

Create React App CLI 工具 是官方支持的建立單頁 React 應用程序的方法。要建一個新的 React 應用程序，請在 VS Code 中打開終端並鍵入以下命令：

```
npx create-react-app react-employee
```

運行這些命令中的任何一個都會在當前文件夾中建一個名為 react-employee 的目錄。

讓我們說明一下 React 項目的重要文件和文件夾。

對於要建立的專案，這些文件必須具有準確的文件名：

public/index.html 是頁面模板；

src/index.js 是 JavaScript 入口點。

您可以刪除或重命名其他文件 讓我們快速瀏覽一下項目結構。

package.json - package.json 文件包含我們的 React JS 專案所需的所有依賴項。最重要的是，您可以檢查您正在使用的 React 的當前版本。它具 React 應用程序的所有腳本。

public 文件夾 - public 文件夾包含 index.html。由於 React 用於建立單頁應用程序，因此我們有一個 HTML 文件來呈現我們所有的組件。基本上，它是一個 HTML 模板。它有一個以 id 為 根的 div 元素，我們所有的組件都在這個 div 中呈現，index.html 作為完整的 React 應用程序的單個頁面。

src 文件夾- 在此文件夾中，我們擁有所有全局 javascript 和 CSS 文件。我們將構建的所有不同組件都放在這裡。

`index.js` - 這是你的 React 应用程序的上層渲染器。

`node_modules` - NPM 或 Yarn 安裝的所有包都將駐留在 `node_modules` 文件夾中。

`App.js` - `App.js` 文件包含我們的 App 組件的定義，它實際上在瀏覽器中呈現，這是根組件。

2 - 使用 NPM 在 React 中添加 Bootstrap

打開一個新的終端窗口，導航到項目的文件夾，然後運行以下命令：

```
npm install bootstrap --save
```

安裝引導程序套件後，您需要將其導入到您的 React 应用程序入口文件中。

打開 `src/index.js` 文件並添加以下代碼：

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

開發 React 組件

Function 組件是我們整個 React 应用程序的建立核心。它們就像接受 `props`、`state` 方面的輸入並輸出在瀏覽器中呈現的 UI 的函數。

React 組件可以是函數組件，也可以是類別組件。在此範例中，我們將使用帶有 Reach Hooks 的功能組件。

讓我們建一個 `EmployeeComponent.js` 文件並向其中添加以下程式碼。

```
import React, {useState, useEffect} from 'react'
```

```
function EmployeeComponent() {
```

```
    const [employees, setEmployees] = useState([])
```

```
    useEffect(() => {  
        getEmployees()  
    }, [])
```

```
    const getEmployees = () => {
```

```
        setEmployees([  
            {  
                "id":1,  
                "firstName":"Ramesh",  
                "lastName":"Fadatare",
```

```

        "email":"ramesh@gmail.com"
    },
    {
        "id":2,
        "firstName":"Tony",
        "lastName":"Stark",
        "email":"tony@gmail.com"
    },
    {
        "id":3,
        "firstName":"John",
        "lastName":"Cena",
        "email":"cena@gmail.com"
    }
  ])
};

return (
  <div className = "container">

    <h1 className = "text-center"> Employees List</h1>

    <table className = "table table-striped">
      <thead>
        <tr>
          <th> Employee Id</th>
          <th> Employee First Name</th>
          <th> Employee Last</th>
          <th> Employee Email</th>
        </tr>

      </thead>
      <tbody>
        {
          employees.map(
            employee =>
              <tr key = {employee.id}>
                <td> {employee.id }</td>

```

```

        <td> {employee.firstName }</td>
        <td> {employee.lastName }</td>
        <td> {employee.email }</td>

    </tr>

    )
    }
</tbody>
</table>

</div>
)
}

```

`export default EmployeeComponent`

讓我們解釋一下上面程式碼片段中使用的 **React Hooks**。

useState() Hook

useState 是一個 **Hook**（函數），它允許您在功能組件中擁有 **state** 變數。您將初始狀態傳遞給此函數，它返回一個具有當前狀態值（不一定是初始狀態）的變數和另一個更新此值的函數。

```
const [employees, setEmployees] = useState([])
```

useEffect() Hook

useEffect Hook 允許我們在函數組件中 **useEffect (action)**。它不使用類別組件中可用的組件生命週期方法。換句話說，**Effects Hooks** 相當於 **componentDidMount()**、**componentDidUpdate()**和 **componentWillUnmount()** 生命週期方法。

useEffect 具有大多數網絡應用程序需要執行的共同特徵，例如：

- 更新 DOM，
- 從服務器 API 獲取和使用數據，
- 設置訂閱等

範例： 更新 DOM：

```
useEffect(() => {  
    getEmployees()  
}, [])
```

```
const getEmployees = () => {
```

```
    setEmployees([  
        {  
            "id":1,  
            "firstName":"Ramesh",  
            "lastName":"Fadatare",  
            "email":"ramesh@gmail.com"  
        },  
        {  
            "id":2,  
            "firstName":"Tony",  
            "lastName":"Stark",  
            "email":"tony@gmail.com"  
        },  
        {  
            "id":3,  
            "firstName":"John",  
            "lastName":"Cena",  
            "email":"cena@gmail.com"  
        }  
    ])
```

```
    // EmployeeService.getEmployees().then((response) => {  
    //     setEmployees(response.data)  
    //     console.log(response.data);  
    // });  
};
```

4. App.js

在上一步中，我們建了 `EmployeeComponent` 所以讓我們繼續將 `EmployeeComponent` 添加 到 `App` 組件：

```
import './App.css';
import EmployeeComponent from './components/EmployeeComponent';

function App() {
  return (
    <EmployeeComponent />
  );
}
export default App;
```

5. 運行 React 應用

使用以下命令啟動項目：

```
npm start
```

在開發模式下運行應用程序。在瀏覽器中打開 `http://localhost:3000` 即可查看。